

Finale

Chris Kauffman

*Last Updated:
Thu May 7 01:35:51 PM EDT 2026*

Logistics

Assignment / Exam Grading

Likely to finish by end of week (Fri/Sat) so folks know scores going into the final Exam

Final Exam

- ▶ Thu Thu 14-May in ESJ 2208 (normal classroom)
- ▶ 10:30am - 12:30pm
- ▶ Comprehensive exam: all topics in the course eligible
- ▶ 6 sides of paper with problems broaching our areas of focus (see later slides for review topics)

Further Coursework

- ▶ **Application areas such as machine learning and data analysis** to see places you might properly apply your parallel skills
 - ▶ CMSC422 Introduction to Machine Learning
 - ▶ CMSC426 Computer Vision
 - ▶ CMSC454 Algorithms for Data Science
 - ▶ CMSC470 Introduction to Natural Language Processing
 - ▶ CMSC472 Introduction to Deep Learning
- ▶ **Science of Computation** to deepen your understanding of fundamental techniques in computation, often arising in HPC
 - ▶ CMSC460 Computational Methods
 - ▶ CMSC466 Introduction to Numerical Analysis I
 - ▶ CMSC660 Scientific Computing I
- ▶ **Hardware** to get further knowledge of the machine that we rely on, be it serial or parallel
 - ▶ CMSC411 Computer Systems Architecture
 - ▶ CMSC662 Computer Organization and Programming for Scientific Computing

Review Topic: Shared Memory Coordination

- ▶ Nearby is a pair of routines which sum random numbers in a sequence
- ▶ On running the code and timing with increasing numbers of threads, the following are observed

#threads	niters	wall time(s)	correct?
serial	10mil	8.05	Yes
omp 1	10mil	8.07	Yes
omp 2	10mil	12.32	No
omp 4	10mil	19.71	No
omp 8	10mil	30.68	No

Explain these and how to fix the parallel version

```
long randsum_serial(int niters){  
    long sum = 0;  
    for(int i=0; i<niters; i++){  
        long num = rand();  
        sum += num;  
    }  
    return sum;  
}
```

```
long randsum_omp(int niters){  
    long sum = 0;  
    #omp parallel  
    {  
        for(int i=0; i<niters; i++){  
            long num = rand();  
            sum += num;  
        }  
    }  
    return sum;  
}
```

Review Topic: GPU Thread Sync

A

We have seen that CUDA provides the `__syncthreads()` function to synchronize threads within a thread block.

Why is this function necessary? Give an example where it would be useful.

B

What are the limits to `__syncthreads()`?

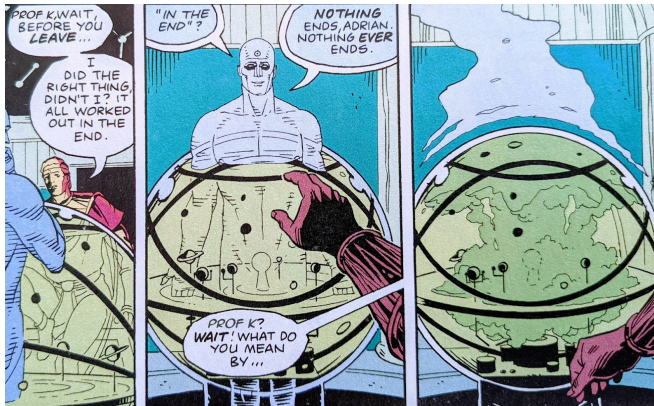
Which threads does it synchronize and which threads does it not?

What alternatives are available for thread synchronization?

Review Topic Requests

- ▶ Distributed Memory Architecture
- ▶ Shared Memory Architecture
- ▶ MPI for Distributed Memory Programming
- ▶ PThreads + OpenMP for Shared Memory Programming
- ▶ CUDA for GPU Programming
- ▶ Communication Patterns
 - ▶ Broadcast, Scatter/Gather, Reductions
 - ▶ Synchronization between procs/threads
- ▶ Input/Output Problem Decomposition
- ▶ Theoretical and Practical Tools for Performance Analysis
- ▶ Parallel Problems/Applications
 - ▶ Sums, Min/Maxes
 - ▶ Matrix Multiplication
 - ▶ Solving Linear Systems
 - ▶ Sorting
 - ▶ Basics of N-Body simulation, Neural Networks, Crypto Mining, Heat Propagation, K-Means

Nothing Ever Ends



By now you should realize that what you learned

- ▶ Will come up again showing whether you learned it well the first time or need another pass.
- ▶ Will change in the future and make you feel old.

Expect this and stay stay patient.

Conclusion

It's been a hell of a semester.
I'm proud of all of you.
Keep up the good work.
Stay safe. Happy Hacking.

